Using Large Language Models to Generate Clinical Trial Tables and Figures

Yumeng Yang, MS[1 2], Peter Krusche, PhD[3], Kristyn Pantoja, PhD[4], Cheng Shi, PhD[1], Ethan Ludmir, MD[5], Kirk Roberts, PhD[2], Gen Zhu, PhD[1*]
[1]Novartis Pharmaceuticals Corporation, East Hanover, NJ, USA
[2]School of Biomedical Informatics
The University of Texas Health Science Center at Houston, Houston, TX, USA
[3]Novartis Pharma AG, Basel, Switzerland
[4]Novartis Pharmaceuticals Corporation, Cambridge, MA, USA
[5]Department of Radiation Oncology
The University of Texas MD Anderson Cancer Center, Houston, TX, USA

*Correspondence to: Gen Zhu, gen.zhu@novartis.com

arXiv:2409.12046v2 [cs.CL] 19 Sep 2024

Abstract

Tables, figures, and listings (TFLs) are essential tools for summarizing clinical trial data. Creation of TFLs for reporting activities is often a time-consuming task encountered routinely during the execution of clinical trials. This study explored the use of large language models (LLMs) to automate the generation of TFLs through prompt engineering and few-shot transfer learning. Using public clinical trial data in ADaM format, our results demonstrated that LLMs can efficiently generate TFLs with prompt instructions, showcasing their potential in this domain. Furthermore, we developed a conservational agent named "Clinical Trial TFL Generation Agent": An app that matches user queries to predefined prompts that produce customized programs to generate specific pre-defined TFLs.

1 Introduction

In the pharmaceutical industry, submission of a clinical study report (CSR) is part of the drug approval process with health authorities. CSRs are highly standardized: For example, ICH (International Council for Harmonisation of Technical Requirements for Pharmaceuticals for Human Use) provides a guideline about the structure and content of the CSR (E3 Structure and Content of Clinical Study Reports — FDA). In the CSR, tables, figures and listings (TFLs) are the essential elements for summarizing clinical trial data, including, for example, summaries of demographic data, efficacy and safety results. Importantly, all analyses and TFLs that summarize outcomes are pre-specified from the protocol and statistical analysis plan through a set of highly detailed documents that are produced before any data are collected. Data shown in each TFL is pre-determined and traceable to data collection, and the layout of TFLs is specified via TFL shells – mockup versions of the table or figure that explicitly determine formatting, labels and footnotes. Once data have been collected, the industry practice to populate the TFLs is based on the data in Study Data Tabulation Model (SDTM) and Analysis Data Model (ADaM) formats which are introduced in 2006 by the Clinical Data Interchange Standards Consortium (CDISC) as data standards for drug submissions (CDISC ADaM)[1].

To implement TFLs, statisticians and statistical programmers need to prepare and validate statistical programs from a data & variable specification/data dictionary and a TFL shell that specifies the layout and calculation of each table or figure or listing. Once these programs and outputs are validated, they can be included in the clinical study report (CSR). This workflow ensures consistency, accuracy, and comprehensibility of the trial data, ensuring effective and un-biased analysis and interpretation. It critically relies on skilled statisticians and programmers to produce each TFL output and is time-consuming due to the large number of outputs, as well as complexity introduced by study-specific pre-specifications, such as specific custom variable mappings, counts and visual representations. Recent advances in large language models (LLMs) have demonstrated substantial potential to accelerate applications that involve text generation, classification, and natural language understanding[2,3,4]. Such LLMs have already shown power in many fields[5,6,7]; their applications in clinical trials have been particularly promising, with LLMs being employed to assist in various tasks, such as the classification of clinical trial eligibility criteria and the extraction of relevant medical information from unstructured data sources. Some research focuses on using language models to extract the eligibility criteria from clinical trial protocols[8,9], while others explore using these models to assess patients' eligibility from

electric health record (EHR)[10,11]. As LLMs have also been successfully applied to link natural language specifications and program code – e.g. by generating programs or test cases[12], using LLMs for statistical programming has received much attention recently[13,14].

In our work, we investigated the use of LLMs for generating TFLs for clinical trial data analysis. We focused on table and figure generation (a listing can be treated as a table) from tabular data. Tabular data are different in structure from the text datasets that LLMs are typically used and trained with. Integrating LLMs into the TFLs generation from tabular data is not straightforward, as the ability to understand table structure and analyze tables using LLMs has not been explored as thoroughly as work with plain text[15,16,17]. Different from plain text documents, tables are structured with complex interrelations between rows and columns. Previously, table reasoning tasks such as table-based question answering[18], table-based fact verification[19] and table-to-text[20] were often tackled by pre-training or fine-tuning neural language models. Recent development of LLMs provide potential to solve these tasks with higher accuracy[15,16,17]. Our TFL use case presented the additional challenge of table-to-table/figure tasks, which may require advanced data manipulation and data analysis. A critical aspect of working with tabular data is enabling the model to understand the relationships between rows and columns, which is essential for accurate data interpretation. Three primary LLM-based approaches from the literature may provide solutions to our problem: 1) fine-tuning open-source LLMs with labeled data; 2) prompting LLMs directly by taking advantage of LLMs' reasoning ability; 3) developing an LLM-based agent that can be integrated with other computing tools such as Python[16,21,22]. In our study, we only considered the second and third approaches.

This work focused on leveraging LLM-based approaches to automate the generation of TFLs that are guided in ICH E3 (E3 Structure and Content of Clinical Study Reports — FDA). More specifically, we investigated the use of LLMs to reproduce the outputs from the CDISC pilot dataset, an open-source dataset for testing tools and methods for clinical trial reporting and analysis[23]. We considered demographic summaries, baseline calculations, and efficacy summaries at different endpoints. More details about the approaches and data are discussed in the Methods section. The Evaluation and Results section discusses the performance of our approach and introduces an app based on the outputs generated by AI that can serve as an example for how this approach may be operationalized. Limitations and future development ideas are discussed at the end.

2 Method

2.1 Data Description

The data in our study is from the CDISC Pilot replication Github repository which aims to reproduce the table outputs within the CDISC Pilot Project using the PHUSE Test Data Factory project's data[24]. This dataset has been used to demonstrate new analytical methods and tools for clinical studies, for example, utilizing open-source software tools such as the R programming language[23].

We started with datasets in ADaM format and aimed to produce TFL outputs from these. The goals of starting with the ADaM data format include easier and more efficient data review, replication, and output generation. The homogeneity of column names in the ADaM format simplifies the expansion to various trials in the future. The data used to generate the table results include the following datasets adhering to CDISC standards: adsl, adadas, adcibc, and adnpix. These datasets provide a comprehensive overview of the clinical trial, covering various dimensions of patient information and assessments on a subject level. The adsl dataset gives subject-level data with one record per subject, capturing demographics, treatment arms, and key study-specific attributes. In addition, adlb, adadas, adcibc, and adnpix provide data on visit-based assessments, offering detailed insights into cognitive function, clinician impressions, and neuropsychiatric symptoms, respectively. Together, these datasets allow for an in-depth analysis of treatment efficacy and patient outcomes throughout the trial. In order to derive summaries for TLF outputs, we typically subset some of these datasets to a specific set of events, visits or variables, joined with adsl for subject-level summaries and then produced listings or counts.

Our method involved leveraging LLMs as agents to generate Python code based on specific prompts or tasks related to data analysis. While R and Python are both among the commonly-used open-source statistical programming languages in the pharmaceutical setting, LLMs have been shown to yield executable code more often for Python than for R[25]. SAS is also a commonly used statistical programming language, however it is proprietary. The model interprets the

user's input, understands the task requirements, and generates the corresponding Python code by calling the LLM API. Once the code is generated, it is automatically executed in a Python interpreter with access to the input datasets to produce the results. This approach streamlines the data analysis process and allows incremental experimentation with the outputs by generating code and output simultaneously. Being able to inspect both code and output helps us to ensure accuracy and transparency in the analysis. Notably, we also experimented with prompting without the intermediate code generation step, having the model populate, e.g., table shells directly. However, we found that the accuracy of the table results was too low to pursue this solution further.

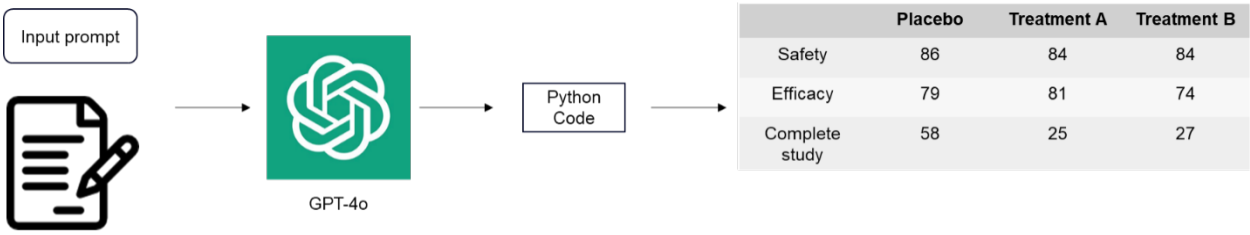


Figure 1: Using the LLM as an Agent to Generate Results

2.2 Prompts Design

LLMs have been shown to be sensitive to the format of prompts[26,27,28]. In our approach, we provided three key components to the model: the system prompt, the user prompt, and a few-shot coding examples. The system prompt defines the model's scope of work; in our case, we set it as, "You are a statistical programming assistant," to focus the model's responses on statistical analysis tasks. The user prompt contains detailed instructions to guide the model, including filtering relevant columns, categorizing data by treatment groups, performing counting calculations, and computing specific statistical measurements such as mean, median, standard deviation (SD), and range. More details on these instructions can be found in table 1.

Additionally, we included some Python code examples in the prompt to help the model understand how to read different formats of inputs files and convert byte strings to regular strings. We also instructed the model to avoid printing intermediate steps and instead directly generate the desired Python code, execute it automatically, and then display the final result. The first prompt was designed to generate Python code to calculate the summary statistics for each treatment arm, with the results shown in Figure 2. The second prompt was intended to generate Python code for plotting the Kaplan-Meier curve for time to the first dermatologic event, with the results displayed in Figure 3. Both sets of generated code matched the manually coded results with 100% accuracy.

<table>
<tr><th>Prompt Example</th><th>Generated Python Code</th></tr>
<tr><td>Please provide Python code to count the number of subjects in each treatment arm and then, within each treatment arm, count the number of subjects for safety (SAFFL), efficacy (EFFFL), Intent-To-Treat (ITTFL), Complete Week 24 (COMP24FL). The code should use <code>pd.read_sas</code> with the <code>format</code> parameter set to <code>xport</code> to import the dataset file if it has an .xpt extension, otherwise use <code>pd.read_csv</code> to import the dataset file if it has a .csv extension.</td><td><pre>
import pandas as pd
# Load the dataset
df = pd.read_sas('adsl.xpt', format='xport')
# Convert byte strings to regular strings
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = df[col].apply(lambda x: x.decode('utf-8')
        if isinstance(x, bytes) else x)
# Count the number of subjects in each treatment arm
treatment_counts = df['ARM'].value_counts().reset_index()
treatment_counts.columns = ['Treatment␣Arm', 'Number␣of␣Subjects']
# Count the number of subjects for each flag within each treatment arm
flags = ['SAFFL', 'EFFFL', 'ITTFL', 'COMP24FL']
flag_counts = df.groupby('ARM')[flags].apply(lambda x:
x.apply(lambda y: (y == 'Y').sum())).reset_index()
# Merge the counts
result = pd.merge(treatment_counts, flag_counts,
left_on='Treatment␣Arm', right_on='ARM').drop(columns='ARM')
# Print the result in table format
print(result.to_string(index=False))
</pre></td></tr>
<tr><td>You are a statistical programming assistant. You need to generate the code for a Kaplan-Meier plot for the time to the first dermatologic event, stratified by treatment arms, based on the data provided. The code should use <code>pd.read_sas</code> with the <code>format</code> parameter set to <code>xport</code> to import the dataset file. Before processing the data, convert byte strings to regular strings. Here is some example code: “ for col in df.columns: if df[col].dtype == object: df[col] = df[col].apply(lambda x: x.decode('utf-8') if isinstance(x, bytes) else x) ” Use the entire dataset provided by the user without creating any sample data. Print the result in table format. Just give me the python code.</td><td><pre>
import pandas as pd
import matplotlib.pyplot as plt
from lifelines import KaplanMeierFitter
# Load the dataset
df = pd.read_sas('adtte.xpt', format='xport')
# Convert byte strings to regular strings
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = df[col].apply(lambda x: x.decode('utf-8')
        if isinstance(x, bytes) else x)
# print the result in table format
print(df)
#create the Kaplan-Meier plot
kmf = KaplanMeierFitter()
#Plotting the Kaplan-Meier curve for each treatment arm
plt.figure(figzie=(10,6))

for treatment in df['TRAT'].unique():
    mask = df['TART']== treatment
    kmf.fit(durations=df[mask]['AVAL'],
    event_observed=1-df[mask]['CNSR'],
    label = treatment)
    kmf.plot_survival_function()

plt.title('Kaplan-Meier␣Plot␣for␣Time␣to␣First␣Dermatological␣Event')
plt.xlabel('Time(days)')
plt.ylabel('Survival␣probability')
plt.legend(title='Treatment␣Arm')
plt.grod(True)
plot.show()
</pre></td></tr>
</table>

Table 1: Prompt examples and codes generated by the model

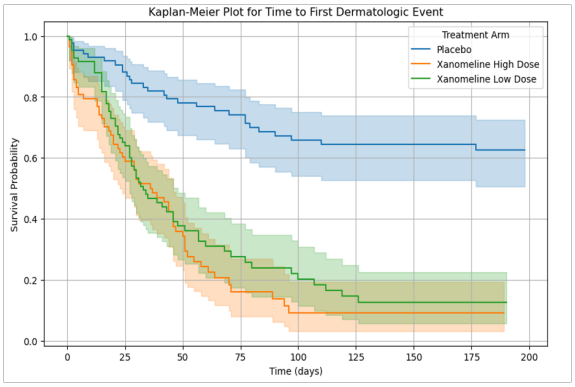
Treatment Arm	Total	SAFFL	EFFFL	ITTFL	COMP24FL	COMPL
Placebo	86 (100.00%)	86 (100.00%)	79 (91.86%)	86 (100.00%)	60 (69.77%)	58 (67.44%)
Xanomeline High Dose	84 (100.00%)	84 (100.00%)	74 (88.10%)	84 (100.00%)	30 (35.71%)	27 (32.14%)
Xanomeline Low Dose	84 (100.00%)	84 (100.00%)	81 (96.43%)	84 (100.00%)	28 (33.33%)	25 (29.76%)
Overall	254 (100.00%)	254 (100.00%)	234 (92.13%)	254 (100.00%)	118 (46.46%)	110 (43.31%)

(a) Table result generated by prompt 1

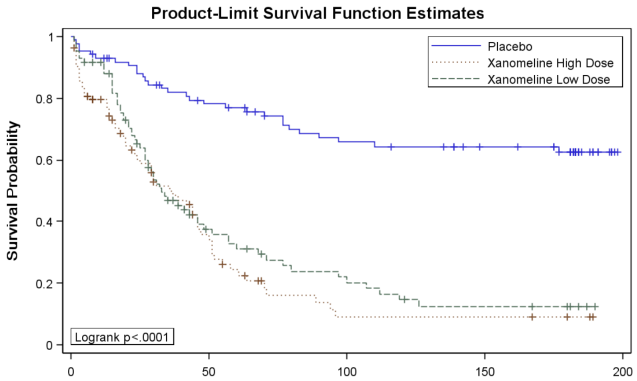
	Placebo (N=86)	Xanomeline Low Dose (N=84)	Xanomeline High Dose (N=84)	Total (N=254)
Intent-To-Treat (ITT)	86 (100%)	84 (100%)	84 (100%)	254 (100%)
Safety	86 (100%)	84 (100%)	84 (100%)	254 (100%)
Efficacy	79 (92%)	81 (96%)	74 (88%)	234 (92%)
Complete Week 24	60 (70%)	28 (33%)	30 (36%)	118 (46%)
Complete Study	58 (67%)	25 (30%)	27 (32%)	110 (43%)

(b) Table generated by manually code

Figure 2: Comparison of table results



(a) Kaplan-Meier plot generated by prompt 2



(b) Original Kaplan-Meier plot

Figure 3: Comparison of Kaplan-Meier plots

3 Evaluation and Result

Our work focused on generating 16 tables and 1 Kaplan-Meier plot. We manually compared all the table results generated by the LLMs with those published in the CDISC pilot replication project, and we also compared the generated Kaplan-Meier plot with the corresponding plot from the study report of the CDISC pilot study[29]. Our designed prompts achieved 100% accuracy in replicating the results, except for analyses involving statistical tests, which need more prompt customization and input from statisticians to define the working steps for the model.

We then tested two predefined prompts for generating tables on a synthetic clinical trial dataset available at Novartis, using its adsl dataset. We attempted to generate two different tables where our prompts were generic enough to also cover a study in a different disease area (the previous study was centered on Alzheimer's disease trials, with some tables specifically tailored to its measurements). Although this new trial had slight differences from our previous one, we were able to reuse the existing prompts with only minor modifications to select the necessary columns; all other aspects remained unchanged. The prompts successfully returned the correct answers, matching the results we manually coded to generate the table outputs.

Our work further demonstrated the feasibility of using large language models like GPT-4o to generate table results from clinical trial ADaM datasets. The 16 prompts we have designed can accurately generate our desired tables. There are multiple endpoints in our example data, including week 8, week 16, and week 24, for each of these endpoint analyses, we use the almost the same prompt but just change the endpoint date, the model can easily identify the difference and return correct results.

4 Dashboard Buildup

The prompts we designed still require some domain and statistical knowledge. To study how we can make the prompts accessible to study team members who may not be familiar with LLMs and prompt design, we have developed our app which we call the Clinical Trial TFL Generation Agent. It is a chatbot-based dashboard that allows users to interactively generate specific clinical trial tables, utilizing a two-layer architecture to ensure precise and relevant responses. When a user makes a request, the model doesn't attempt to guess their intent; instead, it matches the request to one of several predefined prompts, ensuring that the model accurately identifies the information needed and how to present it, regardless of how the request is phrased.

The first layer, called the "Table Category Description," focuses on understanding the user's request by summarizing the overall purpose of the table, such as providing a population summary by different treatment arms. Once the request is clearly understood, the second layer, "Stepwise Detailed Prompts" is activated. This layer contains a fixed set of technical steps necessary to create the table, offering detailed instructions on data categorization, filtering relevant columns, and including Python code examples for tasks like reading different data formats. This structured approach ensures that the data analysis process is efficient, transparent, and easily traceable, as shown in the workflow in Figure 1. The purpose of this design is to minimize the risk of misinterpretation by the model, especially if users provide a vague or incomplete prompt.

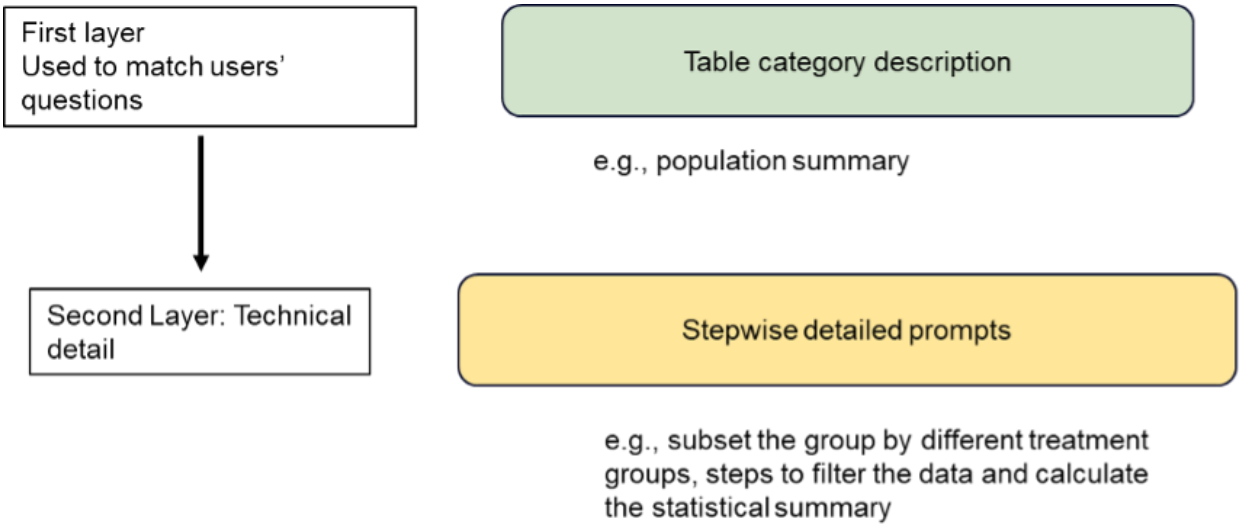


Figure 4: Two-layer architecture of the Clinical Trial TFLs Generation Agent

The backend system matches users' requests with predefined table generation prompts. Here's how it works:

- **Table Generation and Description Matching:** Each table has a detailed description associated with it. When a user makes a request, GPT-4 matches the user's question with the appropriate table description. If a match is found, the system will automatically run the predefined code generation prompts to create the table. The table description part needs close collaboration with statistician by describing the specific subgroup analysis in this table.
- **Non-Code Generation Requests:** Not all user queries require code generation. For requests related to data background information or a general data overview, the app directly returns the relevant description without generating any code. To differentiate between code generation and non-code generation requests, we include a flag after each table description. This flag helps the model identify whether code generation is needed and process the request accordingly.
- **Unmatched Requests:** If a user's request does not match any of the predefined table descriptions, the system will treat the request and their uploaded data as a new prompt, then it will be processed by GPT-4o to generate

the answer, and we added features to allow users to choose if they want the code which used to generated the result, that is helpful for statisticians for future validation and adapt to their own analysis.

This approach ensures that users receive accurate and relevant responses based on their queries, streamlining the table generation process and providing comprehensive data insights efficiently.

5 Discussion and Future Direction

In this project, we have investigated the feasibility of using a large language model to generate clinical trial TFL outputs, and explored a path to implement a user experience that can operationalize this in practice through a TFL Generation Agent application. Unlike general-purpose tools like OpenAI's ChatGPT, our Clinical Trial TFL Generation Agent is designed with a focus on clinical trial calculations, employing a prompt-based design that guides the LLM to produce consistent and accurate results using a standardized prompt library instead of an unguided chat interface. We ensured transparency of the computations performed by outputting generated program code along with the outputs. Our study showed that with careful design of prompts, LLMs be helpful as a tool to search a library of standard TFLs, as well as perform small study-specific adaptations to these prompts. Moreover, standardized prompts can accelerate previewing analysis results and help guide discussions between statisticians, programmers and clinical teams in a data-driven fashion while not increasing the risk of errors that may be introduced by imprecise specifications – and thus could enable question-driven data interrogation similar to Shiny applications[30].

Good performance for generating code and outputs heavily relies on the quality of the predefined prompts, as well as good quality and standards-compliance of the input data. Currently the model struggles with rules requiring complex mapping, coding or tasks requiring advanced/uncommon knowledge to implement the statistical analysis. To overcome this, we would need to develop a prompt library for the TFLs in the CSR that would capture this specialized knowledge, and could supplement existing code templates and macro libraries. We may also be able to improve performance by adding support for creating tables from standardized analysis results datasets[31], which would allow us to further separate programming tasks for data mapping and summarization from data visualization tasks. Additionally, the performance of larger models often improves with increased computational resources, and the raw data from the clinical trial can significantly impact the final results[32,33,34].

Another potential enhancement to our Clinical Trial TFL Generation Agent is to assist with more complex computations. Enhancing the app to provide more precise support in these analyses would be highly beneficial – e.g. also capturing knowledge and pitfalls about method applications in the prompt (but also relying on statistical knowledge built into state of the art LLMs). Finally, capturing interactions between users and LLM-based applications is a good way to gather data to improve responses based on ongoing conversations. This continuous interaction will allow the app to adapt more effectively to user needs and provide increasingly accurate and relevant assistance. This is a much more direct feedback loop than what is typical in existing dashboard or analytical applications with pre-defined visualizations (e.g. R/Shiny) that are commonly used for exploratory data analysis.

In summary, we believe that LLMs can contribute substantially to making the process of TFL generation less manual and more standardized. We believe the key benefit lies in the ability of LLMs to retain a link between program code and natural language / plain text specifications, which allows them to naturally integrate with and enhance the existing processes for designing and producing TFL outputs. The key challenge is in careful prompt design and testing, such that correct outputs can be produced reliably.

Acknowledgement This analysis was carried out when Yumeng Yang interned at Novartis. Yumeng Yang is a Predoctoral Fellow supported by the Cancer Prevention and Research Institute of Texas (CPRIT). We thank our collaborators for their contributions to this work.

Conflict of interest Yumeng Yang has nothing to disclose. Gen Zhu, Peter Krusche, and Kristyn Pantoja report employment by and stock ownership in Novartis.

Author Contributions All authors have reviewed and approved the analysis, contributed to the development and approval of the manuscript, and acknowledged the decision to submit the manuscript for publication.

References

[1] CDISC. CDISC ADaM (Analysis Data Model);. Available from: https://www.cdisc.org/standards/foundational/adam.

[2] Luo R, Sun L, Xia Y, Qin T, Zhang S, Poon H, et al. BioGPT: generative pre-trained transformer for biomedical text generation and mining. Briefings in bioinformatics. 2022;23(6):bbac409.

[3] Yang X, Chen A, PourNejatian N, Shin HC, Smith KE, Parisien C, et al. A large language model for electronic health records. NPJ digital medicine. 2022;5(1):194.

[4] Zhu W, Hu T. Twitter Sentiment analysis of covid vaccines. In: 2021 5th International Conference on Artificial Intelligence and Virtual Reality (AIVR); 2021. p. 118-22.

[5] Wu S, Irsoy O, Lu S, Dabravolski V, Dredze M, Gehrmann S, et al. Bloomberggpt: A large language model for finance. arXiv preprint arXiv:230317564. 2023.

[6] Shin HC, Zhang Y, Bakhturina E, Puri R, Patwary M, Shoeybi M, et al. BioMegatron: larger biomedical domain language model. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP); 2020. p. 4700-6.

[7] Fan X, Tao C, Zhao J. Advanced Stock Price Prediction with xLSTM-Based Models: Improving Long-Term Forecasting. Preprints. 2024 August.

[8] Yang Y, Jayaraj S, Ludmir E, Roberts K. Text Classification of Cancer Clinical Trial Eligibility Criteria. In: AMIA Annual Symposium Proceedings. vol. 2023. American Medical Informatics Association; 2023. p. 1304.

[9] Yang Y, Gilliam A, Ludmir EB, Roberts K. Exploring the Generalization of Cancer Clinical Trial Eligibility Classifiers Across Diseases. arXiv preprint arXiv:240317135. 2024.

[10] Juhn Y, Liu H. Artificial intelligence approaches using natural language processing to advance EHR-based clinical research. Journal of Allergy and Clinical Immunology. 2020;145(2):463-9.

[11] Zeng Z, Deng Y, Li X, Naumann T, Luo Y. Natural language processing for EHR-based computational phenotyping. IEEE/ACM transactions on computational biology and bioinformatics. 2018;16(1):139-53.

[12] Jiang J, Wang F, Shen J, Kim S, Kim S. A Survey on Large Language Models for Code Generation. arXiv preprint arXiv:240600515. 2024.

[13] Coello CEA, Alimam MN, Kouatly R. Effectiveness of ChatGPT in coding: a comparative analysis of popular large language models. Digital. 2024;4(1):114-25.

[14] Communications of the ACM. The Impact of AI on Computer Science Education; n.d. [cited 2024 Sep 16]. Available from: https://cacm.acm.org/news/the-impact-of-ai-on-computer-science-education/.

[15] Zhang X, Wang D, Dou L, Zhu Q, Che W. A Survey of Table Reasoning with Large Language Models. arXiv preprint arXiv:240208259. 2024.

[16] Lu W, Zhang J, Zhang J, Chen Y. Large language model for table processing: A survey. arXiv preprint arXiv:240205121. 2024.

[17] Sui Y, Zhou M, Zhou M, Han S, Zhang D. Table meets llm: Can large language models understand structured table data? a benchmark and empirical study. In: Proceedings of the 17th ACM International Conference on Web Search and Data Mining; 2024. p. 645-54.

[18] Iyyer M, Yih Wt, Chang MW. Search-based neural structured learning for sequential question answering. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers); 2017. p. 1821-31.

[19] Chen W, Wang H, Chen J, Zhang Y, Wang H, Li S, et al. Tabfact: A large-scale dataset for table-based fact verification. arXiv preprint arXiv:190902164. 2019.

[20] Wang Z, Dong H, Jia R, Li J, Fu Z, Han S, et al. Tuta: Tree-based transformers for generally structured table pre-training. In: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining; 2021. p. 1780-90.

[21] Zha L, Zhou J, Li L, Wang R, Huang Q, Yang S, et al. Tablegpt: Towards unifying tables, nature language and commands into one gpt. arXiv preprint arXiv:230708674. 2023.

[22] Zhang T, Yue X, Li Y, Sun H. Tablellama: Towards open large generalist models for tables. arXiv preprint arXiv:231109206. 2023.

[23] Atorus Research. CDISC Pilot Replication: A modern replication of the CDISC pilot table outputs using the PHUSE Test Data Factory ADaM data, using the R programming language;. Available from: https://github.com/atorus-research/CDISC_pilot_replication.

[24] PHUSE. Test Dataset Factory;. Available from: https://advance.phuse.global/display/WEL/Test+Dataset+Factory.

[25] Buscemi A. A comparative study of code generation using chatgpt 3.5 across 10 programming languages. arXiv preprint arXiv:230804477. 2023.

[26] Zhao Z, Wallace E, Feng S, Klein D, Singh S. Calibrate before use: Improving few-shot performance of language models. In: International conference on machine learning. PMLR; 2021. p. 12697-706.

[27] Lu Y, Bartolo M, Moore A, Riedel S, Stenetorp P. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. arXiv preprint arXiv:210408786. 2021.

[28] Liu S, Chen C, Qu X, Tang K, Ong YS. Large language models as evolutionary optimizers. In: 2024 IEEE Congress on Evolutionary Computation (CEC). IEEE; 2024. p. 1-8.

[29] CDISC. CDISC SDTM ADaM pilot study report; 2006. Available from: https://github.com/cdisc-org/sdtm-adam-pilot-project/blob/master/updated-pilot-submission-package/900172/m5/53-clin-stud-rep/535-rep-effic-safety-stud/5351-stud-rep-contr/cdiscpilot01/cdiscpilot01.pdf.

[30] Teal. Teal: An R-Shiny Framework to Unlock the Power of Interactive Data Exploration; n.d. [cited 2024 Sep 16]. Available from: https://teal.posit.co/.

[31] CDISC. Analysis Results Standard;. Available from: https://www.cdisc.org/standards/foundational/analysis-results-standard.

[32] Creswell A, Shanahan M, Higgins I. Selection-inference: Exploiting large language models for interpretable logical reasoning. arXiv preprint arXiv:220509712. 2022.

[33] Fan X, Tao C. Towards resilient and efficient llms: A comparative study of efficiency, performance, and adversarial robustness. arXiv preprint arXiv:240804585. 2024.

[34] Sherry AD, Msaouel P, McCaw ZR, Abi Jaoude J, Hsu EJ, Kouzy R, et al. Prevalence and implications of significance testing for baseline covariate imbalance in randomised cancer clinical trials: The Table 1 Fallacy. European Journal of Cancer. 2023;194:113357.